Merlette — Base de données

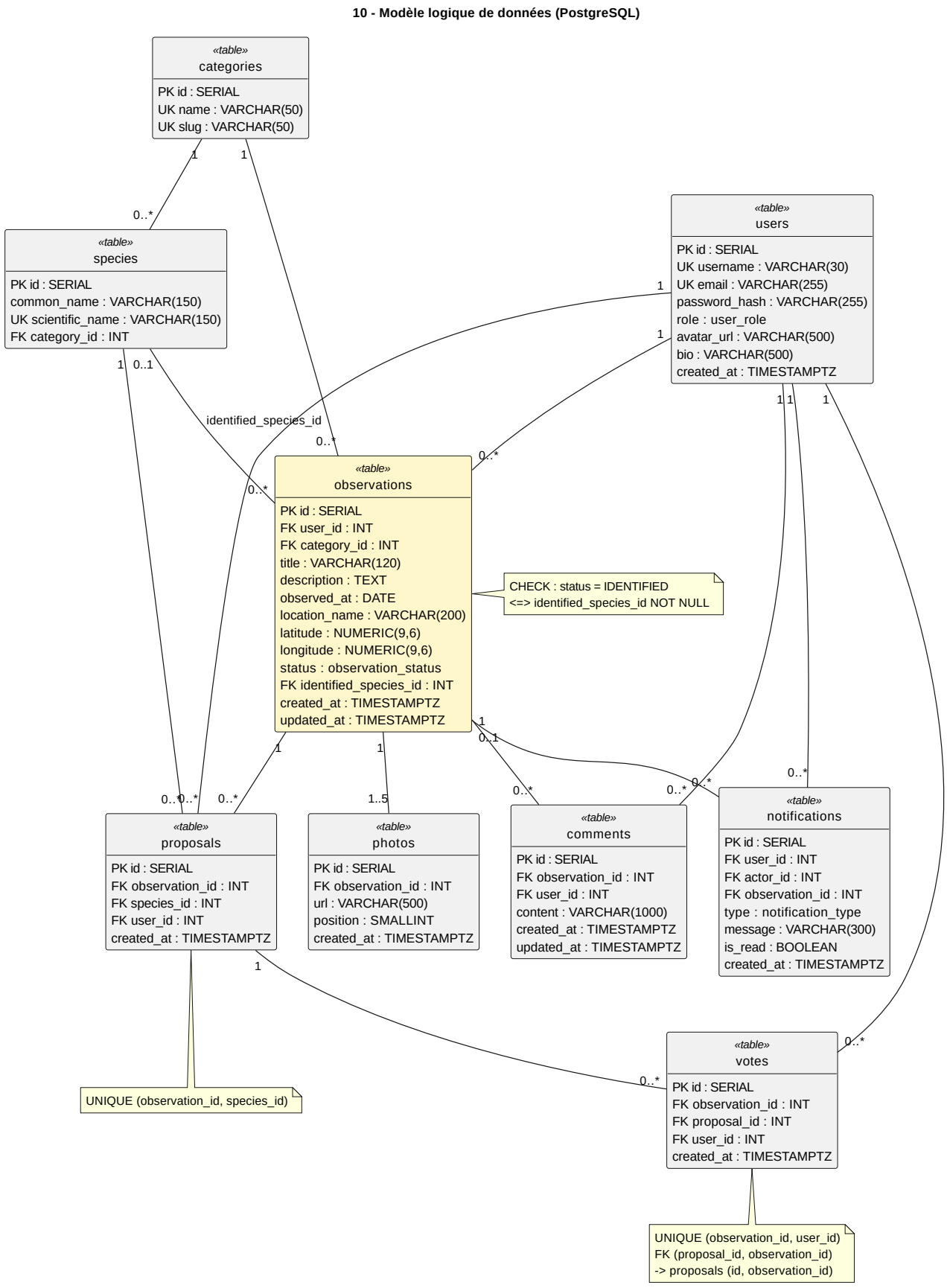
SGBD : **PostgreSQL 14+**. Script complet (tables, contraintes, index, données de référence) : `schema.sql`.

```
createdb merlette
psql -d merlette -f docs/schema.sql
```

Compte admin de démo : `admin@merlette.local` / `Admin1234`.

---

1. Modèle logique de données



Source : `visual_paradigm/mer_10_mld_base_donnees.puml` — explications dans `03-uml.md`.

2. Dictionnaire des données

users

Colonne	Type	Contraintes	Description
id	SERIAL	PK	
username	VARCHAR(30)	NOT NULL, UNIQUE	Pseudo public
email	VARCHAR(255)	NOT NULL, UNIQUE	Identifiant de connexion
password_hash	VARCHAR(255)	NOT NULL	Hash bcrypt
role	user_role	DEFAULT 'USER'	`USER` ou `ADMIN`
avatar_url	VARCHAR(500)		
bio	VARCHAR(500)		
created_at	TIMESTAMPTZ	DEFAULT NOW()	

categories

Colonne	Type	Contraintes	Description
id	SERIAL	PK	
name	VARCHAR(50)	NOT NULL, UNIQUE	Ex. « Oiseaux »
slug	VARCHAR(50)	NOT NULL, UNIQUE	Ex. `oiseaux` (utilisé dans les filtres)

species

Colonne	Type	Contraintes	Description
id	SERIAL	PK	
common_name	VARCHAR(150)	NOT NULL	Nom vernaculaire
scientific_name	VARCHAR(150)	NOT NULL, UNIQUE	Nom latin
category_id	INT	FK → categories, NOT NULL	

observations

Colonne	Type	Contraintes	Description
id	SERIAL	PK	
user_id	INT	FK → users, NOT NULL, CASCADE	Auteur (observateur)
category_id	INT	FK → categories, NOT NULL	
title	VARCHAR(120)		
description	TEXT		
observed_at	DATE	NOT NULL, ≤ aujourd'hui	Date de l'observation sur le terrain
location_name	VARCHAR(200)		Ex. « Parc de la Tête d'Or »

Colonne	Type	Contraintes	Description
latitude / longitude	NUMERIC(9,6)	bornes géographiques	Facultatif
status	observation_status	DEFAULT 'NEEDS_ID'	Recalculé par la règle de consensus
identified_species_id	INT	FK → species, NULL	Rempli **si et seulement si** status = IDENTIFIED (CHECK)
created_at / updated_at	TIMESTAMPTZ	DEFAULT NOW()	

photos

Colonne	Type	Contraintes	Description
id	SERIAL	PK	
observation_id	INT	FK → observations, CASCADE	
url	VARCHAR(500)	NOT NULL	Ex. `/uploads/1697-abc.jpg`
position	SMALLINT	0–4, UNIQUE par observation	Ordre d'affichage (max 5 photos)

proposals

Colonne	Type	Contraintes	Description
id	SERIAL	PK	
observation_id	INT	FK → observations, CASCADE	
species_id	INT	FK → species	
user_id	INT	FK → users	Proposeur
		UNIQUE (observation_id, species_id)	Une espèce proposée une seule fois par observation

votes

Colonne	Type	Contraintes	Description
id	SERIAL	PK	
observation_id	INT	NOT NULL	Dupliqué volontairement pour la contrainte d'unicité
proposal_id	INT	NOT NULL	
user_id	INT	FK → users	Votant
		UNIQUE (observation_id, user_id)	**Un vote par utilisateur et par observation**
		FK (proposal_id, observation_id) → proposals(id, observation_id)	La proposition appartient bien à l'observation

comments

Colonne	Type	Contraintes	Description
id	SERIAL	PK	

Colonne	Type	Contraintes	Description
observation_id	INT	FK → observations, CASCADE	
user_id	INT	FK → users	
content	VARCHAR(1000)	NOT NULL, non vide	
created_at / updated_at	TIMESTAMPTZ		`updated_at > created_at` ⇒ « modifié »

notifications

Colonne	Type	Contraintes	Description
id	SERIAL	PK	
user_id	INT	FK → users, CASCADE	Destinataire
actor_id	INT	FK → users, SET NULL	Qui a déclenché l'événement
observation_id	INT	FK → observations, CASCADE	
type	notification_type	NOT NULL	`NEW_PROPOSAL`, `NEW_COMMENT`, `OBSERVATION_IDENTIFIED`
message	VARCHAR(300)	NOT NULL	Texte affiché
is_read	BOOLEAN	DEFAULT FALSE	

---

3. Requêtes clés

Fil des observations (#8, #9)

```
SELECT o.id, o.title, o.status, o.created_at, c.slug AS category,
       u.username,
       (SELECT url FROM photos p WHERE p.observation_id = o.id ORDER BY position LIMIT 1) AS
cover_url,
       (SELECT COUNT(*) FROM proposals pr WHERE pr.observation_id = o.id) AS proposal_count,
       (SELECT COUNT(*) FROM comments cm WHERE cm.observation_id = o.id) AS comment_count
FROM observations o
JOIN categories c ON c.id = o.category_id
JOIN users u      ON u.id = o.user_id
WHERE ($1::text IS NULL OR c.slug = $1)          -- ?category=
  AND ($2::observation_status IS NULL OR o.status = $2)  -- ?status=
ORDER BY o.created_at DESC
LIMIT $3 OFFSET $4;
```

Votes par proposition (#5) et calcul du consensus (#4)

```
SELECT pr.id, s.common_name, s.scientific_name, pr.species_id,
       COUNT(v.id) AS votes,
       ROUND(100.0 * COUNT(v.id) / NULLIF(SUM(COUNT(v.id)) OVER (), 0), 1) AS percent
FROM proposals pr
JOIN species s ON s.id = pr.species_id
LEFT JOIN votes v ON v.proposal_id = pr.id
WHERE pr.observation_id = $1
GROUP BY pr.id, s.common_name, s.scientific_name, pr.species_id
ORDER BY votes DESC, pr.created_at ASC;
```

Le service prend la 1re ligne : consensus si `votes >= 3` et `votes * 3 >= total * 2`.

Changer de vote (#4)

```
INSERT INTO votes (observation_id, proposal_id, user_id)
VALUES ($1, $2, $3)
ON CONFLICT (observation_id, user_id)
DO UPDATE SET proposal_id = EXCLUDED.proposal_id, created_at = NOW();
```

Mes discussions (#10)

```
SELECT o.id, o.title, o.status, MAX(a.at) AS last_activity
FROM observations o
JOIN (
    SELECT observation_id, created_at AS at FROM comments  WHERE user_id = $1
    UNION ALL
    SELECT observation_id, created_at       FROM proposals WHERE user_id = $1
    UNION ALL
    SELECT id,             created_at       FROM observations WHERE user_id = $1
) a ON a.observation_id = o.id
GROUP BY o.id
ORDER BY last_activity DESC;
```

Nombre de notifications non lues (#6)

```
SELECT COUNT(*) FROM notifications WHERE user_id = $1 AND is_read = FALSE;
```